

System Calls - lseek() and umask()

System Call - lseek()

- Every open file has an associated *offset* value.
- Offset is non-negative integer that measures the number of bytes from the beginning of the file.
- When a file is opened, offset is set to 0.
- Read/write operation starts at an offset and increment the offset by the number of bytes read or written.
- An open file offset can be explicitly repositioned by using `lseek()` system call, based on the value of `whence`:

`SEEK_SET` offset is set to offset bytes from the beginning of the file.

`SEEK_CUR` offset is set to its current value plus the offset.

`SEEK_END` set to the size of the file plus the offset.

```
#include <unistd.h>
// returns new file offset, or -1 on error
off_t lseek(int fd, off_t offset, int whence);
```

lseek() - Code Example

```
#include <fcntl.h>
#include <stdio.h>
#include <stdlib.h>
#include <sys/stat.h>
#include <unistd.h>

/* err_sys display error message and exit */
void err_sys(char *);
int main() {
    int fd, offset;
    if ((fd = open("datafile.dat", O_RDWR | O_CREAT | O_EXCL, S_IRREAD | S_IWRITE)) < 0)
        err_sys("File Open Error");
    if ((offset = lseek (fd, 0, SEEK_END)) == -1)
        err_sys("lseek Error");
    printf("offset = %d\n", offset);
    exit (0);
}

void err_sys(char *str) {
    printf ("%s", str);
    exit (1);
}
```

lseek() - Testing Seek Capability from Standard Input

```
/* this program is to test whether
   the standard input is capable of seeking an offset
*/

#include <fcntl.h>
#include <stdio.h>
#include <stdlib.h>
#include <sys/stat.h>
#include <unistd.h>

int main() {
    if (lseek(STDIN_FILENO, 0, SEEK_CUR) == -1)
        printf("cannot seek.\n");
    else
        printf("Seek OK. \n");
    exit (0);
}
```

lseek() - Testing Size of Input File

```
/*
this program display the size of input file.
*/

#include <fcntl.h>
#include <stdio.h>
#include <stdlib.h>
#include <sys/stat.h>
#include <unistd.h>

int main() {
    int offset;
    if ((offset = lseek(STDIN_FILENO, 0, SEEK_END)) == -1)
        printf("cannot seek.\n");
    else
        printf("file size is %d bytes.\n", offset);
    exit (0);
}

// testing example:
./a.out < datafile.dat
```

lseek() - In-Depth Example

```
// This program creates a file with a hole in it.
#include <fcntl.h>
#include <stdio.h>
#include <stdlib.h>
#include <sys/stat.h>
#include <unistd.h>

char buf1[]="abcdefghij";
char buf2[]="ABCDEFGHIJ";
// err_sys display error message and exit
void err_sys(char *str);

int main() {
    int fd;

    if ((fd = open("datafile.dat", O_RDWR | O_CREAT | O_EXCL, S_IRREAD | S_IWRITE)) < 0)
        err_sys("Open File Error\n");
    if (write (fd, buf1, 10) != 10)
        err_sys("Write buf1 Error\n"); //now offset = 10
    if (lseek (fd, 40, SEEK_SET)== -1)
        err_sys("lseek Error\n"); //now offset =40
    if (write(fd, buf2, 10) != 10)
        err_sys("write buf2 Error\n"); //now offset = 50
    exit (0);
}

void err_sys(char *str) {
    printf ("%s",str);
    exit (1);
}
```

lseek() - In-Depth Example 2

```
// create a file with a hole in it
#include <stdio.h>
#include <stdlib.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <unistd.h>

char buf1[]="abcdefghij";
char buf2[]="ABCDEFGHIJ";
/* err_sys display error message and exit */
void err_sys(char *str) {
    printf ("%s",str);
    exit (1);
}

#define FILE_MODE (S_IRUSR | S_IWUSR|S_IRGRP|S_IROTH)

int main() {
    int fd;

    if ((fd = creat("filehole.txt", FILE_MODE)) <0)
        err_sys("Creat File Open Error");
    if (write (fd, buf1, 10) != 10)
        err_sys("Creat Write Error"); /*now offset = 10*/
    if (lseek (fd, 40, SEEK_SET)== -1)
        err_sys("Creat seekl Error"); /*now offset =40 */
    if (write(fd, buf2, 10) != 10)
        err_sys("Creat write Error"); /*now offset = 50 */

    return 0;
}
```

System Calls - `pread()` and `pwrite()`

- `pread()` is equivalent to the two operations `lseek()`, `read()` together, when they are treated as a single operation.
- `pwrite()` is equivalent to the two operations `lseek()`, `write()` together, when they are treated as a single operation.
- The `pread()` and `pwrite()` system calls are especially useful in multithreaded applications. They allow multiple threads to perform I/O on the same file descriptor without being affected by changes to the file offset by other threads.

System Calls - pread() and pwrite() Prototypes

```
#include <unistd.h>
//A void pointer means it can accept any pointer type
//Returns: size of bytes read from fd, 0 if end of file, -1 on error
ssize_t pread(int fd, void *buf, size_t count, off_t offset);

//Returns: size of bytes written to the fd, 0 if end of file, -1 on error
ssize_t pwrite(int fd, void *buf, size_t count, off_t offset);
```

- `pread()` reads `count` bytes from file descriptor `fd` at `offset` into the buffer starting at `buf`. The file offset is not changed.
- `pwrite()` writes `count` bytes from the buffer starting at `buf` to the file descriptor `fd` at `offset`. The file offset is not changed.
- The file referenced by `fd` must be capable of seeking such as file, stdin, not pipes, nor sockets.

Example Code for pread() and pwrite()

```
#include <fcntl.h>
#include <stdio.h>
#include <string.h>
#include <unistd.h>
#include <sys/stat.h>
#include <sys/types.h>

int main(){
    int fd, nr, nr2, nw, nw2;
    char buf_wr[]={"This is First message to write"};
    char buf_wr2[]={"This is second message to write"};
    char buf_rd[120];
    //open file
    fd = open("out.txt", O_RDWR|O_CREAT, 0666);
    //write first message to the file
    nw = pwrite(fd, &buf_wr, strlen(buf_wr), 0);
    //write the second message just after the first message to the file
    nw2 = pwrite(fd, &buf_wr2, strlen(buf_wr2), strlen(buf_wr));
    //read the second message from the file
    nr = pread(fd, &buf_rd, sizeof(buf_rd), strlen(buf_wr));
    printf("read from file: %s",buf_rd);
    close(fd);
    return 0;
}
```

Managing file permissions

File permissions, especially the default ones, are very important to Linux security and administration.

- Every file or directory has three levels of ownership:
 - User owner (**u**).
 - Group owner (**g**).
 - Others (**o**).
- Each level of ownership can be assigned the following permissions:
 - Read (**r**).
 - Write (**w**).
 - Execute (**x**).
- The first character associated with a file or directory permission denotes the file type, **d** for directory, **-** for file, **l** for symbolic link, etc..
- When a new file or directory is created, the default set of permissions are automatically assigned to it. The default permissions for a file or directory are based on two factors:
 - Base permission.
 - The user file-creation mode mask (**umask**).

Base file permissions

For each level of ownership, base permissions for a file or directory can be expressed in symbolic, octal, or binary values.

Permission	Symbolic Value	Octal value	Binary value
No permission	---	0	000
Execute	--x	1	001
Write	-w-	2	010
Write and execute	-wx	3	011
Read	r--	4	100
Read and execute	r-x	5	101
Read and Write	rw-	6	110
Read, write, execute	rxw	7	111

For example,

`-rw-rw-rw-` means the owner, the group, and all others have read and write permissions only, not execution.

`drwxr-x--x` means the owner has all read, write, and execution rights, the group has read and execution rights, and all others have only the execution right.

Note that the first character denotes file type, `d` for directory, `-` for file, `l` for symbolic link, etc..

User file-creation mode mask (umask)

- The user file-creation mode mask (umask) is a variable that controls how permissions are set for newly created files and directories.
- For directories, the base permissions are `0777` or `rwXrwxrwx` and for files they are `0666` (`rw-rw-rw-`)
- The umask automatically removes its value from the base permission value to increase the overall security of a Linux system.
- The default umask for both a standard user and for a root user is `0022`; the first digit represents special permissions (sticky bit). The umask can be updated by the command such as `umask 0033`.
- Given a mode, say `666`, mask `022`: the result is $666 \& \sim 022$, where $\sim$ means 1's complement. In this case it reduces to $666 \& 755$, which yields `644`.
- For directories, the base permissions are `0777` or `rwXrwxrwx` and for files they are `0666` (`rw-rw-rw-`)

System Call - umask()

- A umask system call set a new value for the process and return the previous value. Here's its prototype:

```
#include <sys/types.h>
#include <sys/stat.h>
// returns previous file mode creation
mode_t umask(mode_t cmask);
```

- Users normally don't deal with umask value.
- When writing a program that create new files, if we want to assure that specific access permission bits are not enabled, we must modify the umask value while the process is running.

Code Example to Modify New File's Base Permission

```
#include <fcntl.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <unistd.h>

int main(){
    int fd;
    fd = open("testonly", O_CREAT, 0777);
    if (fd == -1)
        return -1;
    close(fd);
    return 0;
}
```

After running the default executable ./a.out

```
ls -la testonly
-rwxr-xr-x  1  user  0 Jan 16 19:56 testonly
```

Code Example to Use umask() System Call

```
#include <fcntl.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <unistd.h>
int main() {
    int fd;
    fd = open("test1", O_CREAT, 0777);
    if (fd == -1)
        return -1;
    close(fd);

    umask(0);
    fd = open("test2", O_CREAT, 0777);
    if (fd == -1)
        return -1;
    close(fd);

    umask(S_IWGRP|S_IWOTH);
    fd = open("test3", O_CREAT, 0666);
    if (fd == -1)
        return -1;
    close(fd);
    return 0;
}
```

After running the default executable ./a.out

```
ls -la test*
```

```
-rwxr-xr-x  1 user  staff    0 Jan 16 20:04 test1
-rwxrwxrwx  1 user  staff    0 Jan 16 20:04 test2
-rw-r--r--  1 user  staff    0 Jan 16 20:04 test3
```


Another Code Example to Use umask() System Call

```
//change a file permission inside a process
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <sys/types.h>
#include <sys/stat.h>

void err_sys(char *str) {
    printf ("%s\n",str);
    exit (1);
}

int main () {
    umask(0);
    if (creat ("foo",S_IRUSR |S_IWUSR|S_IRGRP|S_IWGRP|S_IROTH|S_IWOTH) <0)
        err_sys ("creat Error for foo ");

    umask(S_IXGRP|S_IWGRP|S_IXOTH|S_IWOTH); /* same as 0033 */
    if (creat ("bar",S_IRUSR |S_IWUSR|S_IRGRP|S_IWGRP|S_IROTH|S_IWOTH) <0)
        err_sys ("creat Error for bar ");

    exit(0);
}
```